

TCG.Bet

Documentacao Final - Projeto de Software com Arquitetura Justificada

Grupo: preencher nomes dos integrantes | Repositorio: <https://github.com/KachanChK/TCG.Bet>

Resumo executivo

Aplicacao web de apostas em eventos futuros. A melhoria arquitetural adotou monolito modular, camadas inspiradas em Clean Architecture/Hexagonal, ADRs, OpenAPI, testes automatizados e padroes GoF aplicados a problemas reais do dominio.

Sumario

- 1 Introducao
- 2 Atributos de qualidade e decisoes arquiteturais
- 3 Estilo arquitetural
- 4 Aplicacao dos principios SOLID
- 5 Clean Code
- 6 Padroes de projeto GoF
- 7 Design de API
- 8 Diagramas e modelos
- 9 Conclusoes
- 10 Referencias bibliograficas
- 11 Anexos

1. Introducao

O TCG.Bet e um software web para gerenciamento de apostas em eventos futuros. O sistema atende usuarios que desejam criar eventos, apostar, movimentar carteira e acompanhar historico financeiro, alem de moderadores responsaveis por avaliar eventos e finalizar resultados.

A escolha do tema e adequada ao trabalho porque envolve regras de negocio nao triviais: autenticao, autorizacao por papel, operacoes financeiras, historico de carteira, moderacao e calculo de premiacao. Esses pontos permitem demonstrar arquitetura, atributos de qualidade, SOLID, Clean Code e padroes GoF com evidencias objetivas no codigo.

Objetivo geral

Desenvolver e documentar uma aplicacao de apostas em eventos futuros com arquitetura justificada, contrato REST formal e melhoria estrutural do codigo para favorecer manutencao e testabilidade.

Objetivos especificos

- Permitir cadastro, login e autenticao por JWT.
- Permitir criacao, avaliacao, cancelamento e finalizacao de eventos.

- Permitir deposito, saque, aposta e registro de historico financeiro.
- Separar regras de negocio de detalhes de infraestrutura.
- Documentar decisoes arquiteturais por ADRs e especificar a API com OpenAPI.
- Demonstrar SOLID, Clean Code e tres padroes GoF aplicados ao projeto.

Problema e publico-alvo

O problema tratado e a ausencia de uma plataforma simples para concentrar eventos de aposta, validacao por moderadores, participacao dos usuarios e controle de saldo. O publico-alvo sao usuarios adultos interessados em apostas sobre eventos futuros e moderadores que controlam a qualidade e o encerramento dos eventos.

2. Atributos de Qualidade e Decisoes Arquiteturais

2A. Atributos de qualidade prioritarios

Atributo	Justificativa	Decisoes	Metricas
Seguranca	O sistema lida com autenticao, autorizacao e operacoes financeiras.	JWT, AuthPort/JwtOracleAuthAdapter, validacao de papel e hash de senha com script.	Rotas privadas exigem Bearer token; tokens expiram em 8h; senhas novas nao ficam em texto puro.
Confiabilidade e consistencia	Apostas, saldo e historico financeiro precisam permanecer coerentes.	Operacoes financeiras em uma conexao Oracle e commit apenas em sucesso; dinheiro em centavos.	Build sem erros; testes de dominio passando; valores invalidos rejeitados.
Manutenibilidade e testabilidade	Regras misturadas com SQL e Express dificultavam testes.	Camadas domain/application/infrastructure, portas pequenas e padroes GoF.	Casos de uso dependem de interfaces; 9 testes automatizados.

2B. Registro de Decisoes Arquiteturais

Os ADRs ficam versionados na pasta /adrs. O ADR 0005 registra uma decisao modificada: a primeira versao usava servicos com SQL inline; a refatoracao moveu fluxos centrais para casos de uso e portas.

ADR	Titulo	Status	Resumo
0001	Adotar monolito modular	Accepted	Unico deploy com separacao modular.
0002	Organizar por camadas	Accepted	Separa domain, application, infrastructure e HTTP.
0003	REST com OpenAPI	Accepted	Formaliza contrato em docs/openapi.yaml.
0004	Strategy, Factory e Adapter	Accepted	GoF aplicado a problemas reais.
0005	Revisar SQL inline	Accepted	Modifica decisao inicial e melhora testabilidade.

3. Estilo Arquitetural

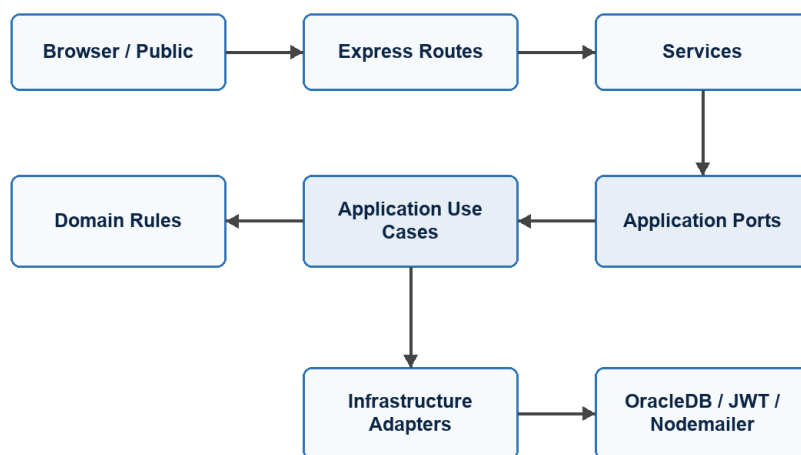
3A. Plano macro

O sistema foi mantido como monolito modular. A escolha é coerente com o tamanho do projeto, com a maturidade esperada de um trabalho acadêmico e com a necessidade de consistência nas operações financeiras. Microserviços aumentariam custo operacional e complicariam transações sem benefício proporcional ao escopo atual.

3B. Plano interno

A organização interna é inspirada em Clean Architecture e Arquitetura Hexagonal. As regras puras ficam em domain; os casos de uso e portas ficam em application; os detalhes de OracleDB e JWT ficam em infrastructure; routes e services fazem a entrada HTTP e preservam compatibilidade com a API existente.

Arquitetura geral - monolito modular em camadas



Fonte versionada: diagrams/architecture.mmd

Figura 1 - Arquitetura geral. Fonte: diagrams/architecture.mmd.

4. Aplicacao dos Principios SOLID

Principio	Explicacao	Trecho aplicado	Efeito
Single Responsibility	Uma classe deve ter uma unica razao para mudar.	MoveWalletFundsUseCase, WalletTransactionFactory e OracleWalletRepository.	Reduz mistura entre regra, SQL e HTTP.
Open-Closed	Aberto para extensao, fechado para modificacao.	FinishEventUseCase depende de PayoutStrategy.	Novo algoritmo de premiacao sem alterar o caso de uso.
Liskov Substitution	Implementacoes substituem a abstracao sem quebrar clientes.	OracleWalletRepository implementa WalletRepository.	Permite repositorio fake em testes.
Interface Segregation	Clientes nao dependem de metodos que nao usam.	WalletRepository, BetRepository, EventRepository e AuthPort.	Contratos pequenos e baixo acoplamento.
Dependency Inversion	Alto nivel depende de abstracoes.	Casos de uso recebem portas no construtor.	SQL, JWT e Oracle ficam fora da regra de negocio.

```
export class FinishEventUseCase {
  constructor(
    private readonly auth: AuthPort,
    private readonly events: EventRepository,
    private readonly bets: BetRepository,
    private readonly wallets: WalletRepository,
    private readonly payoutStrategy: PayoutStrategy
  ) {}
}
```

5. Clean Code

As praticas de Clean Code foram aplicadas principalmente nos fluxos que concentram risco de negocio: carteira, aposta e finalizacao de eventos.

Pratica	Evidencia	Efeito
Nomes claros	MoveWalletFundsUseCase, PlaceBetUseCase, FinishEventUseCase.	A intencao aparece no proprio nome.
Funcoes pequenas	toCents, assertPositiveCents e metodos da Factory.	Regras curtas e testaveis.
Reducao de duplicacao	Deposito e saque usam MoveWalletFundsUseCase.	Evita logicas financeiras duplicadas.
Baixo acoplamento	Casos de uso dependem de portas.	Infraestrutura pode mudar com menor impacto.
Erros explicitos	UseCaseResult com ok/fail.	Fluxos retornam erro controlado.

```
export function toCents(amount: number): number {
  if (!Number.isFinite(amount) || amount <= 0) {
    throw new Error("O valor deve ser maior que zero.");
  }
  return Math.round(amount * 100);
}
```

```
}
```

6. Padroes de Projeto GoF

Strategy - comportamento

Problema: o calculo de premiacao pode mudar. A solucao foi criar PayoutStrategy e uma implementacao PoolPayoutStrategy. Beneficio: extensibilidade. Custo: mais uma abstracao para entender.

```
export interface PayoutStrategy {
  calculatePayouts(winningGuess: string, bets: SettledBet[]): Payout[];
}

export class PoolPayoutStrategy implements PayoutStrategy {
  calculatePayouts(winningGuess: string, bets: SettledBet[]): Payout[] {
    const totalPool = bets.reduce((sum, bet) => sum + bet.betAmountInCents, 0);
    // distribuicao proporcional aos vencedores
  }
}
```

Factory - criacao

Problema: transacoes de carteira precisam ser consistentes quanto a tipo e validacao. A Factory padroniza deposito, saque, aposta e ganho. Beneficio: centralizacao da criacao. Custo: chamada indireta em vez de objeto literal.

```
export class WalletTransactionFactory {
  static deposit(walletId: number, amountInCents: number): WalletTransaction {
    return this.create(walletId, amountInCents, "Deposito");
  }
  static win(walletId: number, amountInCents: number): WalletTransaction {
    return this.create(walletId, amountInCents, "Ganho");
  }
}
```

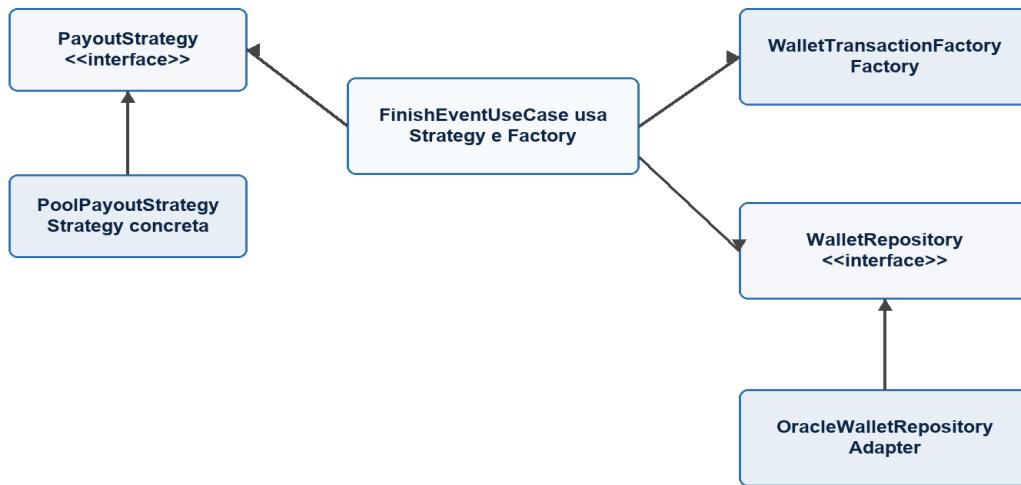
Adapter - estrutura

Problema: casos de uso nao devem depender diretamente de OracleDB, JWT ou bibliotecas externas. Os adapters traduzem essas APIs para portas internas. Beneficio: substituicao e teste. Custo: mais classes de infraestrutura.

```
export class OracleWalletRepository implements WalletRepository {
  constructor(private readonly connection: OracleDB.Connection) {}

  async findById(ownerId: number): Promise<StoredWallet | null> {
    const result = await this.connection.execute(
      `SELECT id, balance, owner_id FROM wallets WHERE owner_id = :ownerId`,
      [ownerId],
      { outFormat: OracleDB.OUT_FORMAT_OBJECT }
    );
  }
}
```

Classes centrais dos padroes GoF



Fonte versionada: diagrams/gof-class-diagram.mmd

Figura 2 - Classes dos padroes GoF. Fonte: diagrams/gof-class-diagram.mmd.

7. Design de API

O estilo escolhido é REST, pois o sistema possui recursos claros como account, wallet, event, bet e moderation. REST combina com Express e com ferramentas de teste e documentação amplamente conhecidas.

- Especificação formal: docs/openapi.yaml em OpenAPI 3.0.3.
- Autenticação: Bearer JWT no header Authorization.
- Versionamento: sem versão explícita na URL nesta iteração; o contrato é versionado pelo campo info.version do OpenAPI.
- Convenção de erro: respostas JSON com propriedade error.
- Paginação: ainda não aplicada; recomendada para endpoints de listagem em uma próxima iteração.

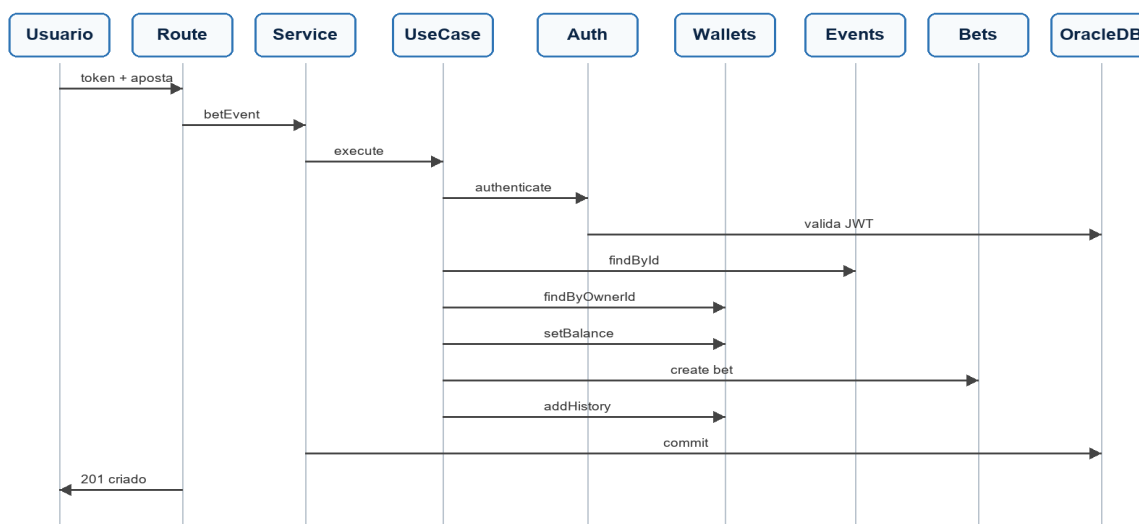
Endpoint	Responsabilidade
POST /account/signUp	Cadastro de usuário.
POST /account/login	Login e emissão de token.
POST /account/addFunds	Deposito em carteira.
POST /account/withdrawFunds	Saque de carteira.
POST /event/addEvent	Criação de evento pendente.
POST /event/bet	Realização de aposta.
POST /mod/evaluateEvent	Aprovação ou reprovção por moderador.

Endpoint	Responsabilidade
POST /mod/finishEvent	Finalizacao e distribuicao de ganhos.

8. Diagramas e Modelos

Os diagramas foram mantidos como código em Mermaid na pasta /diagrams, conforme recomendado no enunciado. Eles incluem visão geral, classes dos padrões GoF e sequência do fluxo central de aposta.

Fluxo central - realizar aposta



Fonte versionada: diagrams/place-bet-sequence.mmd

Figura 3 - Sequencia do fluxo de aposta. Fonte: diagrams/place-bet-sequence.mmd.

9. Conclusões

A refatoração arquitetural tornou o projeto mais defensável: os fluxos centrais passaram a demonstrar separação de responsabilidades, inversão de dependências, padrões GoF aplicados a problemas reais e testes automatizados de domínio.

A prática mais efetiva foi separar regra de negócio de infraestrutura. Isso permitiu testar premiação e transações de carteira sem depender de OracleDB ou servidor Express.

Limitações e próximas melhorias

- Usuários legados com senha antiga devem passar por migração ou troca obrigatória de senha.
- Validação de entrada deveria ser centralizada com Zod, Joi ou class-validator.
- Nem todos os serviços legados foram migrados integralmente para casos de uso.
- Faltam testes de integração para rotas Express e persistência Oracle.
- Os endpoints de listagem ainda não possuem paginação.
- Mensagens e arquivos antigos possuem alguns problemas de encoding em acentos.

- O livro-texto exato da disciplina e os nomes dos integrantes devem ser preenchidos antes da entrega.

10. Referencias Bibliograficas

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1994.

MARTIN, Robert C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River: Prentice Hall, 2008.

MARTIN, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Boston: Prentice Hall, 2017.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. ISO/IEC 25010:2023: Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. Geneva: ISO, 2023.

NYGARD, Michael. Documenting Architecture Decisions. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 10 jun. 2026.

LIVRO-TEXTO DA DISCIPLINA. Referencia a completar conforme bibliografia indicada pelo professor.

Anexos

- Repositorio Git: <https://github.com/KachanChK/TCG.Bet>
- Pasta de ADRs: /adrs
- Pasta de diagramas: /diagrams
- Contrato de API: /docs/openapi.yaml
- README de instalacao e execucao: /README.md
- Testes automatizados: /tests
- Comandos verificados: npm.cmd run build e npm.cmd test